

Rapport de Tests

Jeu vidéo PC : Drive or Die Développeur : [KLAPCZYNSKI / Nathanael] Promotion : [Lagos / 2026] Version : 1.0

1. Technique et démarche de test

La démarche retenue repose sur des **tests fonctionnels manuels**, menés en continu :

- Résultats immédiats et validation visuelle directe en jeu.
- Adéquation avec le temps alloué et le scope d'un prototype d'Infinite Runner.
- Trois niveaux de test :
 1. *Tests au fil de l'eau* : validation de chaque brique dès sa conception (une voiture qui braque, un chunk qui recycle, une entité qui spawn).
 2. *Tests globaux de bout en bout* : parcours complet (menu → partie gagnante → partie perdante → leaderboard).
 3. *Tests aux limites* : comportements inhabituels du joueur, spam d'inputs, collisions multiples, abandon de la partie.

Deux postures complémentaires pendant les tests globaux :

- **Le joueur malveillant** : chercher les failles qui cassent la partie ou faussent le score.
- **Le joueur novice** : vérifier le confort (lisibilité du HUD, compréhension immédiate des règles).

Le développement s'étant déroulé par phases (voiture, génération, entités, combinaison, items, effets, menus, peaufinage), les tests ont suivi le même rythme : chaque phase était testée en jouant avant de passer à la suivante. La phase 3 (conception des entités) est la seule sans test formel : elle était purement conceptuelle.

Les tests aux limites n'ont pas donné lieu à des cas dédiés : ils se sont pratiqués au fil des sessions — spam de touches sur le braquage et la pause, collisions en rafale contre les murs et les entités, sortie de jeu en pleine run. Rien n'en est ressorti au-delà des huit cas listés ci-dessous.

2. Déroulement chronologique des tests

- **Phase 1 — La voiture** : conduite manuelle dans la scène avec de faux chunks. Ce qui était testé : le braquage lissé (le virage ne doit pas être instantané), la réduction de maniabilité avec la vitesse, le realignement du yaw après un coup, et la sensation de vitesse portée par le shake de carrosserie. Deux versions de la direction ont été testées et comparées au volant : pivot proportionnel à la vélocité (rejeté — voir Cas n°3), puis bicycle model. La physique aussi a eu ses corrections : contraintes du Rigidbody contre les déplacements parasites, puis interpolation contre le tremblement sur les bords (voir Cas n°7 et n°8).
 - **Phase 2 — La génération à vue** : tests purement visuels, dans l'éditeur, via les gizmos. Trois itérations testées en conditions réelles de répartition : Perlin (rejeté — performances et rendu), remplissage aléatoire + dilatation, poisson-disque (retenu). C'est le test visuel qui a révélé l'artéfact des colonnes (voir Cas n°2).
 - **Phase 3 — La conception des entités** : pas de test : phase conceptuelle (architecture v1 indépendante, puis v2 ScriptableObjects, pools pensés dès la conception).
 - **Phase 4 — Générateur et entités combinés** : tests du recyclage du ring (le chunk de tête repart-il proprement en queue ?), du re-seeding, et du `RunManager` (ticks, jauges, difficulté). Découverte au passage : la route ne devait pas être dangereuse dès le premier mètre (voir Cas n°4).
 - **Phase 5 — Les items** : tests de collecte : chaque item applique-t-il son effet sur les jauges ? Le pool rend-il bien l'entité après collision ?
 - **Phase 6 — Les effets** : test du feedback : avant la phase, une collision ne disait rien (l'entité disparaissait, c'est tout). Après : effets visuels et sonores à la position de l'entité, qui glissent avec le monde.
 - **Phase 7 — Menus et scores** : flux complet menu → jeu → game over → menu, leaderboard (premier lancement sans fichier, record battu, plafond de cinq entrées), pause et reprise. Au passage : le HUD ne suivait pas la taille de l'écran (voir Cas n°6).
 - **Phase finale — Peaufinage** : test de la **difficulté maximale** — tout est proportionnel en fin de run — qui a révélé le désaccord caméra (voir Cas n°5). Tweaks de cohérence sur le spawn des entités (paramètres de placement ajustés). Équilibrage (vitesse, consommation, budget d'entités), non-régression sur l'ensemble, et tests de performance sur les deux machines avec l'analyse GC au Profiler (voir §5).
-

3. Méthodologie d'analyse et de débbug

Face à une anomalie :

1. **Quand et Comment ?** À quel moment survient le comportement, l'erreur console ou le crash ?
2. **Où ?** Quelles actions ont été réalisées juste avant ? Quel est le processus utilisateur ?
3. **Pourquoi ?** Remonter à la racine du "pourquoi", puis choisir la solution la mieux adaptée.

Quatre familles de correction, selon la nature du bogue :

- L'oubli d'asset : une référence non branchée, une texture ou un matériau manquant — repéré en jouant, corrigé sur le champ. C'est la famille la plus fréquente du prototype : des bugs "neufs" que l'on voit immédiatement dans le jeu.
- La faute de frappe / le mauvais nom de variable (les plus traîtres et les plus difficiles à trouver).
- Le cas non pensé : ajout d'une condition, d'un else, d'un nouveau comportement.
- La logique inappropriée : recoder ou repenser le module concerné.

L'ampleur des re-tests suit l'ampleur de la correction : recoder un module impose de refaire ses tests complets, une correction de variable se vérifie dans son scope.

Outils de débbug utilisés

- **La Console Unity** : erreurs et stack traces en direct. C'est elle qui a désigné `HighScores.Load()` d'emblée dans le cas n°1.
- **Le pas-à-pas dans Visual Studio** : points d'arrêt pour suivre les variables en cours d'exécution, quand la relecture du code ne suffit pas.
- **Le Profiler, sur build** : frame time et allocations GC, sur les deux machines de test (détail au §5).
- **Les gizmos en éditeur** : la génération rendue visible, cœur des tests visuels de la phase 2 (cas n°2).

4. Cas de tests concrets et résolutions de bugs

Cas n°1 : Fichier de leaderboard corrompu

- **Fonctionnalité** : le leaderboard persiste les meilleures runs dans `highscores.json` (chemin de données persistant, plafond de cinq entrées). Le Main Menu le lit (`HighScores.Format()`), le Game Over lit le record (`HighScores.BestScore()`) puis soumet le score (`HighScores.Submit()`) — toutes ces lectures passent par `HighScores.Load()` .
- **Résultat observé (Bug)** : test du cas aux limites "fichier corrompu" : `highscores.json` tronqué volontairement en milieu de fichier (scénario réaliste d'un crash pendant l'écriture), relance du jeu. Le Main Menu casse, la console crache :

```
ArgumentException: JSON parse error: Missing a comma or '}' after an object member.  
UnityEngine.JsonUtility.FromJson[T] (...)  
HighScores.Load () (at Assets/Scripts/HighScores.cs:35)  
HighScores.Format () (at Assets/Scripts/HighScores.cs:48)  
MainMenu.Start () (at Assets/Scripts/MainMenu/MainMenu.cs:12)
```

Par ricochet, le Game Over subirait le même sort : sa lecture du record emprunte le même `Load()` , le score ne serait ni affiché ni enregistré.

- **Cause racine** : `Load()` ne protégeait pas la désérialisation. Un fichier présent mais invalide lève une `ArgumentException` dans `JsonUtility.FromJson` , qui remonte jusqu'au `Start()` de la scène. Le cas "fichier présent mais invalide" n'avait pas été pensé : seul le fichier absent l'était (`Load()` renvoie alors une liste vide).
- **Correction apportée** : famille "cas non pensé : ajout d'une condition" — la lecture est enveloppée dans un try/catch, une désérialisation qui échoue renvoie une liste vide. La dégradation est assumée : le tableau se vide, et le prochain `Submit` réécrit un fichier valide — le système se répare tout seul.

```
// A corrupted file must not break every read: a parse failure resets
// the board, the next Submit rewrites a fresh file
try
{
    return JsonUtility.FromJson<Data>
(System.IO.File.ReadAllText(path)).Entries;
}
catch (Exception)
{
    return new List<Entry>();
}
```

- **Validation (scénarios re-testés après correction) :**
 - *Cas 1.1* : fichier tronqué + relance → le Main Menu affiche un tableau vide, aucune erreur console. → Validé.
 - *Cas 1.2* : prochaine run terminée → `highscores.json` réécrit en fichier valide, nouveau score enregistré, flag "(new)". → Validé.
 - *Cas 1.3* : premier lancement, aucun fichier → liste vide, la première run est enregistrée. → Validé.
 - *Cas 1.4* : score inférieur au record → entrée ajoutée, pas de flag "(new)". → Validé.
 - *Cas 1.5* : score supérieur au record → flag "(new)", tableau réordonné. → Validé.

Cas n°2 : Colonnes visibles dans la répartition des entités

- **Fonctionnalité** : la génération d'entités par poisson-disque (dart throwing sur une grille de cellules) doit produire une répartition naturelle de la route, sans amas ni alignements perceptibles.
- **Résultat observé (Bug)** : pendant la phase 2, en testant visuellement la génération via les gizmos, les éléments placés semblaient posés sur une grille : des colonnes d'entités alignées étaient visibles.
- **Cause racine** : le tirage uniforme dans une grille régulière, même aléatoire, laisse les cellules d'une même colonne à fréquence élevée — l'œil accroche l'alignement. Le placement était correct par entité, mais la structure de la grille transparaissait.
- **Correction apportée** : chaque rangée reçoit un décalage latéral aléatoire d'au plus une cellule, et la colonne du bord, côté décalage, est vidée pour que rien ne

dépasse de la route :

```
// Placement : each row shifts sideways by up to one cell, the edge column
// in the shift direction is dropped so nothing overhangs
for (int row = 0; row < chunkRows; row++)
{
    float rowOffset = cellSizeX * Random.value - cellSizeX * 0.5f;
    if (rowOffset > 0f)
    {
        _grid[row * chunkCols + chunkCols - 1] = null;
    }
    else
    {
        _grid[row * chunkCols] = null;
    }
    // ... puis jitter dans la cellule, réduit de Border
}
```

- **Validation (scénarios re-testés après correction) :**
 - Cas 2.1 : plusieurs chunks générés à la suite → plus aucun alignement en colonnes perceptible. Validé. *(capture d'écran à glisser ici au montage final)*
 - Cas 2.2 : aucune entité ne dépasse de la route, quelle que soit la valeur du décalage. Validé en jeu.
 - Cas 2.3 : deux entités voisines ne sont jamais plus proches que $2 \times \text{Border}$. Validé en jeu.

Cas n°3 : Sensation de glisse de la voiture (direction "en crabe")

- **Fonctionnalité** : la direction doit être cohérente avec la géométrie du véhicule : la voiture pivote comme une voiture, elle ne glisse pas latéralement.
- **Résultat observé (Bug)** : dans la première version, l'input gauche/droite posait directement la vitesse latérale, et la rotation suivait proportionnellement la vitesse (normalisée à sa valeur maximale). En jeu, la voiture donnait l'impression de rouler en crabe : elle glissait sur le sol, le mouvement était incohérent avec le visuel. Feedback au volant immédiat : sensations de gameplay immondes.
- **Cause racine** : le pivot était proportionnel à la vitesse latérale, sans aucun lien avec la géométrie du véhicule (angle des roues, empattement). Rien ne liait la rotation du châssis à la trajectoire des roues : le châssis pivotait sans roue qui braque — la voiture dérapait visuellement.

- **Correction apportée** : refonte du modèle de direction (famille "logique inappropriée : repenser le module") :
 1. Un bicycle model simplifié relie la géométrie au mouvement : vitesse angulaire maximale = $\text{ForwardSpeed} \times \tan(\text{angle roue max} : 35^\circ) / \text{empattement} (2,5 \text{ m})$.
 2. L'angle courant est amené vers sa cible par `SmoothDampAngle`, borné par cette vitesse angulaire.
 3. La vélocité latérale devient une *conséquence* de l'angle : $\text{ForwardSpeed} \times \tan(\text{angle courant})$.
 4. En bonus, l'intégration temporelle du braquage — calculée à la main, en tenant compte de l'accélération du volant qui tourne de plus en plus vite — est remplacée par `SmoothDampAngle`, trouvée après coup en l'utilisant ailleurs dans le projet. Le résultat n'est pas mathématiquement identique à 100 %, mais la différence est imperceptible à l'œil nu, et le code devient standard et lisible.
- **Validation (re-test au volant, version actuelle)** :
 - *Cas 3.1* : braquage → la voiture pivote et glisse dans le virage, la sensation "en crabe" a disparu.
 - *Cas 3.2* : le braquage reste lissé : pas de virage instantané sur un tap.
 - *Cas 3.3* : plus on va vite, moins on est maniable (angle de braquage maximal $25^\circ \rightarrow 12,5^\circ$).
 - *Cas 3.4* : après une collision, le yaw décroît vers l'angle de braquage (`RealignSpeed` = 8) : la voiture reprend sa trajectoire.

Cas n°4 : Dégâts pris avant le premier contrôle

- **Fonctionnalité** : la run doit démarrer en sécurité : le joueur a le temps de prendre ses marques avant sa première menace.
- **Résultat observé (Bug)** : en jouant les premières runs, des cas où le joueur se prenait du dégât immédiat au spawn — avant même d'avoir pu bouger.
- **Cause racine** : le générateur remplissait d'emblée tous les chunks, y compris ceux juste devant la voiture au lancement. Le dart throwing place selon la difficulté : une entité proche du spawn restait possible dès la première seconde.
- **Correction apportée** : les deux premiers chunks (`_emptyChunksAtStart`) sont générés sans entités — le compteur `_filledChunks` marque la zone de départ :

```
for (int i = 0; i < AheadChunks; i++)  
{  
    NewChunk(CreateChunk(), _filledChunks < _emptyChunksAtStart);  
}
```

- **Validation (re-test au lancement) :**
 - *Cas 4.1* : au spawn, la route est dégagée sur les premiers chunks — première entité rencontrée après un temps de réaction confortable. Validé en jeu.
 - *Cas 4.2* : le recyclage suivant remplaçant le premier chunk vide en queue avec entités, la protection ne s'applique qu'au démarrage. Validé en jeu.
-

Cas n°5 : Caméra désaccordée en difficulté maximale

- **Fonctionnalité** : en fin de run (difficulté maximale), la vue doit rester lisible — le monde accélère, la caméra doit suivre.
 - **Résultat observé (Bug)** : en testant la difficulté maximale (tout étant proportionnel en fin de partie), la caméra montrait ses limites : seule la FOV avait été réglée, la position ne suivait pas — la vue était désaccordée en fin de course.
 - **Cause racine** : paramétrage incomplet : la FOV courait seule sur sa courbe, sans sa contrepartie de position.
 - **Correction apportée** : le `CameraManager` couple position et FOV sur la même difficulté — une position initiale et une finale (`(0, 6, -12)` → `(0, 9, -20)`), une FOV initiale et une finale (`60°` → `70°`) — les deux interpolent ensemble.
 - **Validation (re-test en difficulté maximale) :**
 - *Cas 5.1* : du début à la fin de run, la vue reste stable et lisible, sans effet de resserrement ou d'ouverture disproportionnés. Validé en jeu.
-

Cas n°6 : HUD mal mis à l'échelle

- **Fonctionnalité** : l'interface suit la taille de l'écran : jauges et score gardent leurs proportions quelle que soit la résolution.
- **Résultat observé (Bug)** : au premier montage du HUD, l'interface ne scalait pas avec la taille de l'écran — proportions cassées au changement de résolution.
- **Cause racine** : le Canvas n'était pas réglé pour suivre la résolution.
- **Correction apportée** : le Canvas Scaler passe en mode **Scale With Screen Size** : l'UI se cale sur une résolution de référence et s'étire proportionnellement.

- **Validation (re-test à différentes résolutions) :**
 - *Cas 6.1* : barres et score gardent leurs proportions du plein écran au fenêtré. Validé en jeu.
-

Cas n°7 : Rigidbody de la voiture sans contraintes

- **Fonctionnalité** : la voiture est un corps physique guidé : elle ne doit réagir qu'aux forces prévues (yaw du braquage et glisse latérale).
 - **Résultat observé (Bug)** : aux premiers essais, toucher les bords latéraux de la route (les murs qui empêchent de sortir) décalait bizarrement la voiture — elle bougeait anormalement au contact.
 - **Cause racine** : oubli des **contraintes** du Rigidbody : le corps physique restait libre sur tous les axes, un choc pouvait donc le faire pivoter ou déplacer hors du modèle de jeu.
 - **Correction apportée** : les contraintes gèlent tout ce qui n'est pas le jeu prévu — la rotation limitée à l'axe Y (le yaw), la position avant gelée : le monde porte l'avancement, la voiture ne fait que glisser latéralement.
 - **Validation (re-test au volant) :**
 - *Cas 7.1* : percuter un bord latéral → la voiture glisse le long du mur, sans pivot ni décalage anormal. Validé en jeu.
 - *Cas 7.2* : le realignement du yaw (`RealignSpeed`) continue de ramener la trajectoire après le choc.
-

Cas n°8 : Tremblement de la voiture sur les bords

- **Fonctionnalité** : les contacts contre les bords latéraux doivent être visuellement propres.
- **Résultat observé (Bug)** : remarqué après le fix des contraintes : quand le Rigidbody fonce dans les murs latéraux, la voiture **tremble** — la physique clignote entre contact et déplacement, le rendu vibre.
- **Cause racine** : le moteur physique et le rendu tournent à des cadences différentes : la position physique calculée à cadence fixe est rendue telle quelle, sans compensation — le choc répété contre le mur se voit en vibration.
- **Correction apportée** : l'**interpolation** du Rigidbody : la voiture rendue est interpolée entre deux pas physiques — le mouvement redevient continu à l'œil, le tremblement disparaît.

- **Validation (re-test contre les bords) :**
 - *Cas 8.1* : rouler collé au bord latéral à pleine vitesse → plus de vibration visible. Validé en jeu.
-

5. Tests de performance

- **Deux environnements de test** : le PC principal de développement, et un vieux portable (i5 de 6e génération, deux cœurs, près de dix ans) servant de cible bas de gamme. L'objectif : repérer les gros coups de lag et vérifier que le jeu reste fluide sur la machine faible. Le projet est versionné en Git sur un serveur NAS, ce qui rend le test sur les deux machines direct.
 - **Analyse du Garbage Collector** : le cœur du sujet. L'architecture est pensée pour ne plus allouer après le lancement — pooling des entités et des effets, ring de chunks recyclé, grille de placement et delta de collision réutilisés. **Résultat** : **aucun souci** — la promesse a tenu, le développement avait été conçu autour dès le départ ; le Profiler ne relève pas d'allocation récurrente en jeu.
 - **Résultat marquant — le décor** : le décor (arbres, herbe, jusqu'aux poules) était d'abord posé en prefabs placés au hasard. Sur la machine bas de gamme, on descendait sous le seuil de jouabilité retenu (60 FPS), parfois bien moins. Le décor est repassé sous le **Terrain** d'Unity (les entités de gameplay restent des prefabs poolés) : herbe et environnement en masse, fluidité retrouvée sur la vieille machine.
 - **Chiffres du banc (vieux portable, build)** : frame time total moyen de **17 ms** — autour de 59 FPS — et les pires **0,1 %** de frames ne dépassent jamais **25 ms** (plancher à ~40 FPS). L'objectif tient : pas de lag spike, le jeu reste fluide sur la machine bas de gamme.
-

6. Bilan

- Les tests ont fait évoluer le jeu en continu : la direction a été refondée, la physique de la voiture contrainte puis lissée, la répartition corrigée, le spawn protégé, la caméra réaccordée, le HUD scalé, le décor repassé sous le Terrain.
- Plusieurs features sont nées de tests : le système d'effets (une collision muette), la protection du spawn (dégât avant le premier contrôle), l'auto-réparation du JSON (fichier corrompu). Les tests aux limites et le test en conditions réelles ont porté leurs fruits.

- Le Garbage Collector, pensé dès la conception, n'a pas posé de problème : la promesse "zéro allocation après le lancement" a tenu.
- Points restés ouverts : l'inventaire d'items tenus est architecté — les trois hooks (`OnInventory` , `OnScore` , `OnEntityBlueprint`) existent et sont lus — mais jamais alimenté dans le prototype, aucun code n'ajoute d'entité à la liste ; l'équilibrage reste une première passe, à retravailler largement.